

Entrega 1

Alumno : Juan Pablo Arroyo Godinez, Demien Becerra lozano

Materia: Laboratorio de modelado de datos



**ITESO, Universidad
Jesuita de Guadalajara**

Fecha: 24/10/2025

1. Dataset de clusterizacion

Gran parte del desarrollo y decisiones del preprocesamiento se encuentran dentro del notebook

1.1. Problema

Tomamos una base de dato de reviews de amazon de diferentes caregorias de productos, la idea es que con las variables de entrada podamos predecir la categoria del producto

1.1.1. Target

La variable target presenta 4 nivele, cada nivel siendo una categoria de producto de amazon, las cuales presentan un desvalance no tan significativo, quizas no es necesario hacerle nada a la hora de realizar el modelo.

1.1.2. Tratamiento de variables

La variable que encontrabamos mas interesante era evidentemente la variable de `text` y la variable de `title` bajo la primiscia de que en estas variables se encuentra informacion contundente que nos ayude a separar las variables. Cosa que se confirma *at a glance* al ver el head de nuestros datos, vemos que las reviews contienen palabras clave tales como “*cd*” o “*music*”.

el preprocesamiento del texto es el objeto principal de esta entrega, se hicieron otras cosas como dropear un par de variables y alguna que otra trivialidad, por lo que vamos a centrarnos en explicar que fue lo que hicimos con el texto.

El pipeline puede reducirse a los siguientes pasos:

1. Normalizacion de texto
2. Sanitizado de texto
3. Tokenizacion
4. eliminacion de stop words
5. Lemmatizacion
6. Embedding (*en proceso*)

todo el proceso desde los pasos 1 a 5 fueron hechos con la libreria nltk

cuyo outcome es el texto lemmatizado, el texto lemmatizado quiere decir que cada palabra es reducida a su raiz, y elminando de cierta forma sufijos, nexos y otras palabras que nos generan ruido para poder entender el significado de una frase.

Apartir de aqui tenemos varias opciones *BoW*, *TFiDF encoding* o *embbeding*

se decidio optar por la ultima

Existen varias opciones de embedding, word2vec (*embedding por palabra*), por N-gramas por documento o por cadenas de texto completas.

El embedding es la manera en lo que los LLMs encodean palabras manteniendo su significancia semantica. Nuestro problema nos demandaba hacer el embedding las cadenas completas de strings.

generar embeddings de cadenas de texto completas se hace con redes y es un proceso sumamente costoso. por lo tanto se suelen utilizar apis de modelos que nos generan estos embeddings por nosotros, modelos de openai y de gemini. Optamos por utilizar la api de la segunda

1.1.2.1. El inconveniente

al tener un tier gratuito de la api de gemini, estamos limitados a hacer el embedding de 100 palabras por call con 30 calls por minuto, es decir 3000 cadenas de texto por minuto, si intentamos hacer mas la api nos bloquea y no nos permite hacer el embedding.

por lo que despues de procesar 3000 palabras tenemos que esperar 60 segundos para poder volver a hacerlo. resultando en mas de 2 horas para poder procesar tan solo la columna text. No obstante tambien de forma inesperada podemos llegar a sobrecargar la api y resultando en un error 500, que es por parte del servidor y no por parte del lado del cliente. lo que complico poder tokenizar este texto de inicio a final

2. Regresion

2.0.1. Desarrollo inicial

A continuación describirémos lo que hicimos en esta primera parte del proyecto.

Como parte inicial del proyecto hicimos un glosario en donde describimos el contenido de cada columna. ¿Por qué?... Bueno consideramos importante poder describir las columnas con el propósito de tener un mayor entendimiento del dataset, muchas veces los nombres de las columnas ya tienen mucha información sin embargo es que la descripción de estas columnas nos puede dar más detalle de la misma y al menos en este proceso es muy importante tener la mayor comprensión de lo que estaremos trabajando.

Ahora si empezamos bien, empezamos cargando nuestro dataset(en donde guardamos los datos en una variable llamada df, con ayuda de la librería pandas) e importando las librerías necesarias para comenzar con el proceso.

Como parte de nuestro análisis exploratorio realizamos ciertas visualizaciones específicas del dataset como `pd.head()` para observar un poco de los datos que contienen nuestras columnas, después usamos la función `df.info()` que nos da el tipo de datos de nuestras columnas (¿por que es importante?, Bueno dependiendo el dataset podríamos tener variables que deberían de ser de un tipo y por algun factor del vaciado de los datos podría llegar a tener un formato equivocado ejemplo: si tuvieramos una columna de precio y esta tiene signos como . o “,” las puede llegar a considerar como variable objeto en vez de un int o float como debería de ser), también usamos `df.describe()` en donde nos da estadísticos básicos de las columnas numericas, desde un conteo de los datos (en el conteo podemos ver desde ahí que datos nos faltan), también la media, la desviación estandar, el valor min, el max, etc.. esto aunque no lo parezca es muy útil, podemos por ejemplo sacar de ciertas columnas información que nos habla de los datos, supongamos que tenemos una variable llamada edad y tenemos en min -15 y en max 150, desde ahí ya sabemos que tenemos valores atípicos, que probablemente tendríamos que borrar en este caso al menos ya que son cantidades que no son posibles.

Biene lo bueno...

2.0.2. Variables numéricas

Variable “Quantity” En general, para esta variable empecé checando los nulos, no tenía, después fuimos a las gráficas en donde notamos cantidades negativas tanto en el boxplot, con atípicos que pues no deberían de estar, y en la distribución notamos esa afectación gracias a las cantidades negativas. La decisión fue eliminar las filas donde la Quantity fuera menor a 0. Esto se hizo porque en un contexto de ventas, una cantidad negativa generalmente indica una devolución o una

cancelación. Sin embargo, dado que `ReturnStatus` ya maneja la información de devoluciones (“Returned” o “Not Returned”), y las cantidades negativas en `Quantity` y `UnitPrice` (visto en `df.head()`) sugieren transacciones que no son ventas estándar, decidimos eliminarlas para enfocarnos en las transacciones de venta positivas.

Después de eliminar las 2489 filas con valores negativos en `Quantity`, volvimos a revisar la distribución, confirmando que la limpieza fue exitosa. Las nuevas gráficas (histograma y boxplot) reflejaron una distribución solo de valores positivos.

Variable “UnitPrice” Continuamos con el `UnitPrice` (precio por unidad). Al revisar los estadísticos (`df.describe()` antes de limpiar `Quantity`), notamos que esta variable también contenía valores negativos (mínimo de -99.98), lo cual es preocupante por las mismas razones que `Quantity`. Al ser transacciones de devolución, la eliminación de los valores negativos en `Quantity` también eliminó las filas con valores negativos en `UnitPrice`, lo cual era esperado ya que las devoluciones a menudo tienen precios reflejados como negativos. La distribución resultante para `UnitPrice` ahora muestra valores estrictamente positivos.

Variable “Discount” Para la variable `Discount` (descuento aplicado), verificamos la existencia de nulos y encontramos cero nulos. Sus estadísticos (`df.describe()`) mostraban un rango entre 0.00 y 1.99. Al analizar las gráficas, especialmente el boxplot, se observaron valores atípicos superiores a 1.0, llegando hasta un máximo de 1.99.

La `value_counts()` reveló que los descuentos más frecuentes están entre 0.01 y 0.49, con una distribución relativamente uniforme, pero también hay un número menor de registros con 0.00 (486) y 0.50 (493). Sin embargo, los valores atípicos superiores a 1.0 (como el 1.501433 en `df.head()`) son anormales ya que un descuento generalmente se representa como una fracción entre 0 y 1. Decisión: Los valores atípicos superiores a 1.0 deberían eliminarse o tratarse como errores, ya que un descuento superior al 100% no tiene sentido en el contexto de una venta. Procederemos a eliminar o imputar estos valores.

Variable “ShippingCost” La variable `ShippingCost` (costo de envío) fue revisada para buscar nulos. Encontramos 2489 valores nulos. Sus estadísticos (`df.describe()`) indican que la media es de 17.49 y tiene un rango de 5.00 a 30.00. Las gráficas muestran una distribución con un aspecto relativamente simétrico sin valores atípicos extremos, con un pico en la media.

Decisión: Dado que la distribución es simétrica y no presenta una cola larga ni valores atípicos que sesguen la media, la mejor opción para manejar los nulos es la imputación por la media o la mediana. Optaremos por la media.

2.0.3. Variables categóricas

Las variables categóricas, al tener un número manejable de categorías, se decidió que serían transformadas mediante One-Hot Encoding (`pd.get_dummies()`) para ser utilizadas en el modelo de regresión.

Variable “Country” Esta variable no tiene nulos. Contiene 12 países diferentes y la cantidad de registros por país está bien distribuida, con cada país teniendo alrededor de 4000 a 4200 transacciones.

Variable “PaymentMethod” Esta variable no tiene nulos. Tiene 3 métodos de pago (“Bank Transfer”, “Credit Card”, “paypal”) con una distribución muy balanceada, alrededor de 16500 a 16700 transacciones por método.

Variable “Category” La variable Category no tiene nulos. Presenta 5 categorías de producto (“Furniture”, “Accessories”, “Electronics”, “Stationery”, “Apparel”). La distribución es muy uniforme, con cada categoría teniendo entre 9800 y 10000 transacciones.

Variable “OrderPriority” OrderPriority no tiene nulos. Contiene 3 niveles de prioridad (“Medium”, “High”, “Low”) y la distribución es equilibrada, con cada nivel cerca de 16500 a 16700 transacciones.

Variable “Description” Description no tiene nulos. Aunque tiene muchas descripciones, la value_counts() muestra que son 11 productos únicos. La cantidad de registros para cada descripción es similar, alrededor de 4400 a 4600 transacciones.

Variable “SalesChannel” SalesChannel no tiene nulos. Tiene 2 canales de venta (“Online” e “In-store”) con una distribución uniforme, cada uno con alrededor de 24700 a 25000 transacciones.

Variable “ReturnStatus” ReturnStatus no tiene nulos. Contiene 2 estados (“Not Returned” y “Returned”). La gran mayoría de las transacciones son “Not Returned” (44888), con solo 4894 “Returned”. Esta variable puede ser crucial para el modelo.

Variable “ShipmentProvider” ShipmentProvider no tiene nulos. Presenta 4 proveedores (“FedEx”, “UPS”, “DHL”, “Royal Mail”) con una distribución muy uniforme, alrededor de 12400 a 12500 transacciones por proveedor.

Variable “WarehouseLocation” WarehouseLocation inicialmente tenía 3485 nulos. Los valores no nulos se distribuyen entre 5 ciudades (“Amsterdam”, “London”, “Rome”, “Berlin”, “Paris”).

Decisión: Los nulos fueron tratados imputando con la nueva categoría “Desconocido”, ya que la ubicación del almacén podría ser una información relevante que no se quiere perder, y la falta de ella por sí misma puede ser una característica. La value_counts() final incluye la categoría “Desconocido”.

—

2.1. Transformación general

Luego de revisar una por una, podemos validar la decisión de transformarlas a One-Hot Encoding (la cantidad de categorías es suficiente), por lo que podemos aplicarle el one-hot.

Se seleccionaron todas las columnas categóricas (PaymentMethod, SalesChannel, Category, ReturnStatus, ShipmentProvider, WarehouseLocation, OrderPriority, Country, Description) y se aplicó pd.get_dummies() con drop_first=True para evitar la multicolinealidad, creando el DataFrame df_encoded.

—

2.2. Análisis de correlación y selección de variables

Se generó un mapa de calor (sns.heatmap()) para visualizar las correlaciones entre las variables numéricas: InvoiceNo, Quantity, UnitPrice, CustomerID, Discount, y ShippingCost.

2.2.1. Conclusiones del Mapa de Calor

Quantity y UnitPrice muestran una correlación débil de 0.16.

Quantity y Discount presentan una correlación muy débil de -0.0031 .

Quantity y ShippingCost tienen una correlación débil de 0.13 .

Las variables con mayor correlación son UnitPrice y Discount con -0.28 .

Decisión: Como la variable objetivo es Quantity, la correlación lineal de las variables numéricas con ella es débil, lo que sugiere que las variables categóricas (que ahora son dummy variables en df_encoded) podrían ser más influyentes, o que la relación es no lineal. Las variables InvoiceNo, StockCode, CustomerID, e InvoiceDate son identificadores o temporales, por lo que se pueden descartar o requerir un procesamiento adicional.